

BillSense

Deep Technical Analysis & System Audit

Domain	Philippine currency counterfeit detection
Platform	Android (Java) + Vue admin dashboard + FastAPI ML backend
Backend	Google Cloud Run (Asia Southeast 2)
ML Stack	YOLOv8 OBB - 6 specialized models
Firebase Project	bill-sense-aec6b
Report Generated	2026-05-11

Prepared by Claude Code (claude-opus-4-7) - automated codebase audit covering architecture, security posture, deployment health, and remediation paths.

1. Executive Summary

BillSense is a counterfeit-banknote detection system targeting Philippine peso bills. An Android client captures imagery, sends it to a FastAPI service hosted on Cloud Run, which runs an ensemble of six YOLOv8 OBB models to score authenticity against denomination, security features, optically variable ink (OVI), optically variable device (OVD), enhanced value panel (EVP), and direct counterfeit classification. Results route into Firebase RTDB and Storage. A Vue 3 / Vite admin panel provides operational visibility, APK build automation via a host-side Node dev-server, and a Puppeteer-backed GitNexus agent for repository workflows.

Top findings

- **Production dashboard offline:** <https://billsense.dev-environment.site/> returns the default Phusion Passenger placeholder. The Vue build was never deployed; the cPanel Node app stub shadows it.
- **Local ML container not runnable as-is:** the docker-compose API service pulls from a private GCP Artifact Registry that needs interactive gcloud auth login, plus model weights and a Firebase service-account key that are not in the repo.
- **Cloud Run backend is healthy** but reports `models_loaded: false` on cold health checks - models lazy-load on first scan.
- **No CI/CD existed** before this audit. A GitHub Actions workflow has been added to deploy the admin panel to cPanel via FTPS on every push to main.
- **Secret hygiene concerns:** Firebase Web API key embedded in source, `MAPS_API_KEY` in `local.properties`, no `.env` scaffold for `VITE_` keys.

2. System Architecture

Three-tier system: a thick Android client, a stateless Cloud Run inference API, and a Firebase data plane shared across mobile and admin surfaces. An operator-facing Vue dashboard and two helper services (dev-server, GitNexus agent) sit outside the user data path and exist purely for developer ergonomics.

2.1 Component map

Component	Tech	Runtime	Purpose
Android app	Java 11, minSdk 24, AGP 8.1.1	11th devices	Capture, scan, history, FCM, account, chatbot
FastAPI backend	Python, FastAPI v17, OpenCV 4.6.0	Cloud Run	REST + WebSocket inference, 6-model ensemble
Admin panel	Vue 3.5, Vite 6, Firebase 11	Static SPA (cPanel)	Health, GitNexus, APK ops, app testing
Dev server	Node 20 native (no deps)	Host process, port 3003	ADB, Gradle build, APK distribute
GitNexus agent	Express + Puppeteer	Docker, port 3002	Headless GitHub clone automation
Firebase	RTDB + Storage + Messaging	Google managed	Scan history, app config, FCM topics

2.2 Data flow

Scan path: Android camera frame → JPEG base64 over WSS to /ws/standard-scan or REST to /api/standard-scan → FastAPI loads models on demand → OBB detection per model → numbered annotation overlay → result + annotated image to Firebase Storage, summary to RTDB → client renders verdict.

Admin path: Browser SPA → Firebase Web SDK reads RTDB directly → UI panels poll Cloud Run /api/health, localhost:8080 (local container), localhost:3003 (dev-server), localhost:3002 (gitnexus), GitHub API. No backend-for-frontend layer.

3. Android Client

113 Java source files across 9 packages under com.app.billsense. Build configured for Java 11, ViewBinding (no DataBinding, no findViewById), CameraX, OkHttp/Retrofit for network, Glide for image loading, Material Design components.

3.1 Package structure

Package	Responsibility
activities	Top-level screens: Login, Home, Detection, Profile, Cases, Compare, Education, Chatbot, AddVoting
adapters	RecyclerView adapters for scan history, evidence, cases lists
api	Retrofit interfaces and request/response DTOs
fcm	Firebase Cloud Messaging receivers and topic management
fragments	Tab fragments inside Home / Dashboard
interfaces	Listener and callback contracts
model	POJOs for users, scans, evidence, cases
scan	CurrencyApiService, RealTimeScanManager (WebSocket), image preprocessing
utils	Permission helpers, image utils, formatters, validators

3.2 Key activities

- **MainActivity / DispatchActivity** - splash and routing to login or home.
- **LoginActivity** - Firebase Auth, country-code picker, FCM token registration.
- **HomeActivity** - tabbed shell hosting fragments for scan, history, profile.
- **DetectionActivity** - the core scan flow; uses CameraX preview, frame capture, base64 encode at 80% JPEG quality, sends over WebSocket for real-time multi-bill scanning.
- **CompareBillActivity** - side-by-side comparison view for educational training.
- **EducationalContentActivity** - bundled materials on security features.
- **EvidenceActivity / CasesActivity** - field-officer workflow for filing counterfeit incidents.
- **ChatBotActivity** - the Billy AI agent surface for user Q&A.

3.3 Critical scan integration

```
scan/pojo/CurrencyApiService.java
```

REST client wrapping standard-scan, multi-scan, video-scan. Multipart upload, configurable timeouts.

```
scan/pojo/RealTimeScanManager.java
```

OkHttp WebSocket client. Auto-starts on connection (no START_SCAN handshake). Pushes base64 JPEG frames at the configured cadence and parses incoming detection payloads onto the main thread for overlay rendering.

4. ML Inference Backend

FastAPI service deployed to Cloud Run (asia-southeast2). Single-process, thread-pool of 4 workers, CORS open (allow_origins=[*]). Six YOLOv8 OBB models loaded lazily on first request, with a dummy-model fallback for resilience in dev.

4.1 Endpoints

Method	Path	Purpose
POST	/api/standard-scan	One-shot single-image authenticity scan
POST	/api/multi-scan	Detect and score multiple bills in one image
POST	/api/video-scan	Frame-by-frame scan of an uploaded video
GET	/api/health	Liveness, model load state, Firebase reachability
GET	/api/real-time-status	Active WebSocket connection metrics
WSS	/ws/standard-scan	Real-time single-bill stream
WSS	/ws/real-multi-scan	Real-time multi-bill stream
WSS	/ws/real-video-scan	Real-time video frame stream

4.2 Model ensemble

Model file	Role	Output classes (abridged)
denomination2.pt	Bill value classifier	20, 50, 100, 200, 500, 1000
security_best.pt	Security feature detector	watermark, security thread, serial number, value, concealed value, see-through
ovi.pt	Optically variable ink	OVI present / absent / suspicious
ovd.pt	Optically variable device	OVD foil regions
evp.pt	Enhanced value panel	500/1000 EVP variants, false-positive guard
counterfeit_best.pt	Direct counterfeit classifier	UV-thread, missing markers, anomalies

4.3 Notable internals

- Custom JSON encoder for numpy types prevents serialization crashes but converts large arrays unguarded - potential DoS surface.
- ConnectionManager tracks WebSocket lifecycle and assigns numeric IDs; client-supplied client_id is not validated.
- Numbered-annotation system maps detections to feature numbers 1-9 for overlay rendering.
- Firebase init is optional - if service account is missing, a DummyFirebaseClient stub takes over and scans run without persistence.

5. Admin Dashboard

Single-page Vue 3 application. Five views, Firebase Web SDK for direct RTDB and Storage reads, polling-based health checks against the various backends. Builds to static files in dist/ served by nginx in Docker, or by cPanel after deploy.

5.1 Views and routes

Route	View	What it does
/	Dashboard.vue	Connection health tiles for the four core service surfaces
/connection-health	ConnectionHealth.vue	Deep service view: Firebase, GCP, Docker, Mobile, GitNexus, GitHub, Claude Co
/gitnexus	GitNexus.vue	Trigger Puppeteer auto-clone, manage AI keys
/apk-management	ApkManagement.vue	Build / list / download APKs via dev-server
/app-testing	AppTesting.vue	Run scripted ADB checks against the live emulator

5.2 Service modules

- services/firebase.js - initializes the Firebase app with hard-coded web config.
- services/healthCheck.js - probes Cloud Run /api/health, localhost:8080, GitHub, Firebase RTDB, and a synthesized claude-code skill set.
- services/modelConnectionCheck.js - exercises Firebase ML model accessibility, GCP storage, mobile emulator presence.

5.3 Backend dependencies the SPA expects

URL	Purpose	Status (audit run)
http://localhost:3000	Admin SPA itself	OK (docker)
http://localhost:3002	GitNexus agent /api/auto-clone	OK (docker)
http://localhost:3003	Dev-server APK/build/distribute	OK (host node)
http://localhost:8080	Local Docker ML API	OFFLINE - needs gcloud auth
Cloud Run /api/health	Production ML inference	OK, models lazy
bill-sense-aec6b RTDB	Firebase Realtime Database	OK
billsense.dev-environment.site	Production dashboard	PLACEHOLDER - not deployed

6. Deployment & CI/CD

6.1 Current state

- **Mobile:** Manual gradle assembleDebug/installDebug from BillSense/. Firebase App Distribution wiring exists in dev-server but is unauthenticated.
- **Cloud Run API:** Image is pushed to asia-southeast2-docker.pkg.dev/bill-sense-aec6b/billsense/billsense-api:latest. No automation found - assumed manual gcloud run deploy.
- **Admin panel:** No automation before this audit. cPanel was hosting the Phusion Passenger default page because the Vue build was never uploaded.
- **GitNexus / dev-server:** Local-only; no remote deploy needed.

6.2 New CI/CD: admin -> cPanel

Added .github/workflows/deploy-admin-cpanel.yml. On every push to main that touches the admin panel, GitHub Actions installs deps, injects VITE_ env vars, runs npm run build, writes a SPA-aware .htaccess for cPanel/Apache, and pushes dist/ via FTPS using SamKirkland/FTP-Deploy-Action@v4.3.5.

Required secrets and variables

Kind	Name	Example
Secret	CPANEL_FTP_HOST	ftp.dev-environment.site
Secret	CPANEL_FTP_USER	cpaneluser@dev-environment.site
Secret	CPANEL_FTP_PASS	your cPanel password
Variable	CPANEL_REMOTE_DIR	/public_html/billsense/
Variable	VITE_GITNEXUS_REPO	UserDevAccount1/Billsense-Project
Secret	VITE_GEMINI_API_KEY	optional - GitNexus AI
Secret	VITE_OPENAI_API_KEY	optional - GitNexus AI
Secret	VITE_GITHUB_PAT	optional - GitNexus auto-clone

6.3 Why FTPS instead of an MCP

There is no production-grade Model Context Protocol server for cPanel today. cPanel exposes UAPI and WHM API over HTTP Basic Auth - building a custom MCP wrapper is feasible but high-effort and not the right abstraction for a single deploy job. GitHub Actions with FTPS is the standard pattern that every cPanel-hosted SPA uses, runs unattended, and integrates with the rest of the GitHub-native developer flow.

6.4 cPanel cutover checklist

- Stop and remove the Node.js app in cPanel → *Setup Node.js App*. Otherwise Phusion Passenger continues to shadow the static files.
- Confirm the document root for billsense.dev-environment.site under cPanel File Manager.

- Create an FTP account scoped to that document root (cPanel → FTP Accounts).
- Add the secrets and variables above to the GitHub repo.
- Push a no-op change to BillSense Admin/admin-panel/ or trigger the workflow manually.
- Hit <https://billsense.dev-environment.site/> - should render the Vue dashboard.

7. Security Posture

Severity	Finding	Action
High	CORS allow_origins='*' on FastAPI; any origin can call inference endpoints	Restrict to known Android package + admin domain
High	Firebase service-account fallback to DummyFirebaseClient silently disables persistence	Fail disable persistence gate on env
High	VITE_ secrets shipped to browser if set (Gemini, OpenAI, GitHub PAT)	Use proxy through a backend, never ship third-party keys to the S
Med	WebSocket client_id is user-supplied and unvalidated	Allowlist regex, length cap
Med	MAPS_API_KEY stored in local.properties, easy to leak	Move to gradle secrets plugin or build env
Med	Hard-coded Firebase Web API key in services/firebase.js	Acceptable for Web SDK if domain restrictions are set in GCP
Low	Puppeteer runs with --no-sandbox in gitnexus-agent	Acceptable inside docker; document trust boundary
Low	Numpy JSON encoder serializes unbounded arrays	Cap response size; reject oversize tensors

8. Operational Gaps

- **No automated tests in CI.** Android Espresso tests exist but no workflow runs them.
- **No monitoring on Cloud Run.** No Cloud Logging filters, no alerting on error rate or cold-start failures.
- **Models are not versioned.** All six .pt files live alongside the image with no manifest of training run / dataset hash.
- **service_account.json is a placeholder in the repo** - any environment that depends on it will silently fall back to DummyFirebaseClient.
- **No rollback strategy documented** for either Cloud Run or cPanel deploys.
- **Two API base URLs in use** (asia-southeast2 and asia-east1 legacy). Risk of split-brain if both stay live.

9. Recommendations

9.1 Immediate (this week)

- Land the new cPanel workflow, add the secrets, retire the Phusion Passenger default app, confirm the dashboard renders.
- Move VITE_GEMINI_API_KEY / VITE_OPENAI_API_KEY / VITE_GITHUB_PAT behind a tiny proxy (Cloud Functions or a small Express endpoint) so they are never embedded in a public bundle.
- Restrict FastAPI CORS to the Android app's deployed origin set and the admin domain.
- Decide between the asia-southeast2 and asia-east1 endpoints; retire the loser.

9.2 Near term (this month)

- Add a Cloud Run deploy workflow (separate from cPanel one) that builds the API image with Cloud Build and runs gcloud run deploy on tag.
- Wire Espresso and basic Python pytest into a CI workflow gating PRs.
- Publish a model card per .pt file: dataset, training run hash, evaluation metrics.
- Make the Firebase service-account requirement explicit - fail startup in prod if missing.

9.3 Longer term

- Replace Firebase RTDB with Firestore or Cloud SQL if reporting / analytics needs grow (RTDB is fine for FCM-style fanout, not for analytics).
- Move model loading out of process to a TorchServe / Triton sidecar so the API stays warm and scaling is decoupled from inference.
- Add structured logging across the stack (Cloud Logging, FCM event log, scan-result audit trail).
- Treat the admin panel as a privileged tool: add Firebase Auth gating and IAM-style claims before the dashboard exposes operational controls beyond read-only health.

10. Artifacts Produced by This Audit

Path	Purpose
.github/workflows/deploy-admin-cpanel.yml	GitHub Actions workflow that builds and FTPS-deploys the admin panel to cPanel
.github/workflows/README.md	cPanel cutover guide and secret setup checklist
docker/docker-compose.yml	Admin context path restored; billsense-api removed from admin depends_on
pdf-output/BillSense_Deep_Analysis.pdf	This document
pdf-output/generate_report.py	ReportLab source for the document - rerunnable

End of report.